

JavaScript Array Methods

Complete Cheatsheet

...

```
const numbers = [1, 2, 3, 4, 5];
```

```
numbers
```

```
  .filter(n => n > 2)
```

```
  .map(n => n * 2)
```

```
  .reduce((a, b) => a + b);
```

Swipe for more >>

[JD]

| CODING WITH JD

Mutator Methods

Methods that modify the original array

.push()

Adds elements to end of array

```
const arr = [1, 2];  
arr.push(3);  
→ [1, 2, 3]
```

.unshift()

Adds elements to start of array

```
const arr = [1, 2];  
arr.unshift(0);  
→ [0, 1, 2]
```

.pop()

Removes last element

```
const arr = [1, 2, 3];  
arr.pop();  
→ [1, 2]
```

.shift()

Removes first element

```
const arr = [1, 2, 3];  
arr.shift();  
→ [2, 3]
```

.splice()

Add/remove elements at position

```
const arr = [1, 2, 3];  
arr.splice(1, 1, 'two');  
→ [1, 'two', 3]
```

.reverse()

Reverses array in place

```
const arr = [1, 2, 3];  
arr.reverse();  
→ [3, 2, 1]
```

Accessor Methods

Methods that return a new value

`.join()`

Creates string from array elements

```
['a', 'b', 'c'].join('-');  
→ "a-b-c"
```

`.slice()`

Returns portion of array

```
[1, 2, 3, 4].slice(1, 3);  
→ [2, 3]
```

`.concat()`

Merges two or more arrays

```
[1, 2].concat([3, 4]);  
→ [1, 2, 3, 4]
```

`.indexOf()`

Finds first matching element index

```
[1, 2, 1].indexOf(1);  
→ 0
```

`.lastIndexOf()`

Finds last matching element index

```
[1, 2, 1].lastIndexOf(1);  
→ 2
```

`.includes()`

Checks if element exists

```
[1, 2, 3].includes(2);  
→ true
```

Iteration Methods

Methods for looping through arrays

`.forEach()`

Executes function for each element. No return value.

```
[1, 2].forEach(x => {  
  console.log(x * 2);  
});  
→ // logs: 2, 4
```

`.map()`

Creates new array with results of callback.

```
[1, 2, 3].map(x => x * 2);  
→ [2, 4, 6]
```

`.filter()`

Creates array with elements that pass test.

```
[1, 2, 3].filter(x => x > 1);  
→ [2, 3]
```

`.reduce()`

Reduces array to single value (accumulator).

```
[1, 2, 3].reduce(  
  (sum, x) => sum + x, 0  
);  
→ 6
```

`.some()`

Tests if any element passes condition.

```
[1, 2, 3].some(x => x > 2);  
→ true
```

`.every()`

Tests if all elements pass condition.

```
[1, 2, 3].every(x => x > 0);  
→ true
```

Search Methods

Finding Elements in Arrays

`.find()`

Returns first element that passes test

```
[1, 4, 2, 5].find(x => x > 3);  
→ 4
```

`.findIndex()`

Returns index of first matching element

```
[1, 4, 2, 5].findIndex(x => x > 3);  
→ 1
```

`.findLast()`

Returns last element that passes test

```
[1, 4, 2, 5].findLast(x => x > 3);  
→ 5
```

`.findLastIndex()`

Returns index of last matching element

```
[1, 4, 2, 5].findLastIndex(x => x > 3);  
→ 3
```

`.indexOf()`

Returns first index of exact match

```
[1, 2, 2, 3].indexOf(2);  
→ 1
```

`.lastIndexOf()`

Returns last index of exact match

```
[1, 2, 2, 3].lastIndexOf(2);  
→ 2
```


Transform Methods

Array Transformation & Flattening

`.flat()`

Creates new array with flattened sub-arrays

```
[[1,2], [3,4]].flat();  
[1, [2, [3]]].flat(2);  
→ [1, 2, 3, 4] [1, 2, 3]
```

`.flatMap()`

Maps each element and flattens result

```
[1, 2].flatMap(x => [x, x*2]);  
→ [1, 2, 2, 4]
```

`.from()`

Creates array from array-like object

```
Array.from("hello");  
Array.from([1,2], x => x*2);  
→ ['h','e','l','l','o'] [2,  
4]
```

`.of()`

Creates array from variable arguments

```
Array.of(1, 2, 3);  
Array.of('a', null, true);  
→ [1, 2, 3] ['a', null, true]
```

`.fill()`

Fills elements with static value

```
new Array(3).fill(0);  
[1,2,3].fill('a', 1);  
→ [0, 0, 0] [1, 'a', 'a']
```

`.copyWithin()`

Copies array elements to another position

```
[1,2,3,4,5].copyWithin(0,3);  
// target=0, start=3  
→ [4, 5, 3, 4, 5]
```

Ordering Methods

Array Sorting & Reversing

.sort()

Sorts elements in place as strings by default

```
['c', 'a', 'b'].sort();  
[10, 2, 1].sort();  
→ ['a', 'b', 'c'] [1, 10, 2]
```

.sort(compareFn)

Custom sort with compare function

```
[10, 2, 1].sort((a, b) => a - b);  
['b', 'a'].sort((x, y) => x > y ?  
1 : -1);  
→ [1, 2, 10] ['a', 'b']
```

.reverse()

Reverses array elements in place

```
[1, 2, 3].reverse();  
['a', 'b', 'c'].reverse();  
→ [3, 2, 1] ['c', 'b', 'a']
```

Numeric Sort

Proper number sorting with compare function

```
[10, 2, 21].sort();  
[10, 2, 21].sort((a, b) => a - b);  
→ [10, 2, 21] [2, 10, 21]
```

Object Sort

Sort array of objects by property

```
const users = [{age: 30}, {age: 20}];  
users.sort((a, b) => a.age - b.age);  
→ [{age: 20}, {age: 30}]
```

Case Insensitive

Sort strings ignoring case

```
['B', 'a', 'C'].sort((a, b) =>  
  a.toLowerCase() > b.toLowerCase()  
  ? 1 : -1);  
→ ['a', 'B', 'C']
```

Utility Methods

Helpful Array Operations

`.isArray()`

Check if value is an array

```
Array.isArray([1, 2]);  
Array.isArray('hello');  
→ true false
```

`.at()`

Get element at index (supports negative)

```
[1, 2, 3].at(-1);  
[1, 2, 3].at(1);  
→ 3 2
```

`.with()`

Returns new array with replaced element

```
[1, 2, 3].with(1, 'two');  
→ [1, 'two', 3]
```

`.keys()`

Returns array of indices

```
[...['a', 'b'].keys()];  
→ [0, 1]
```

`.entries()`

Returns array of [index, element] pairs

```
[...['a'].entries()];  
→ [[0, 'a']]
```

`.toSorted()`

Returns new sorted array

```
[3, 1, 2].toSorted();  
→ [1, 2, 3]
```